countDigits.1.js

```
// 1)Count the number of digits from a given number(N) --- Brute Force Approach (Extraction Algo)

const countDigits = (n) => {
  let count = 0;
  while (n > 0) {
    // let lastDigit = n % 10;
    count += 1;
    // Math.floor is essential to avoid the loop to run indefinately due to decimal numbers on dividing the value by 10.
    n = Math.floor(n / 10);
  }
  return count;
};
const digits = countDigits(7231);
console.log(digits);
// T.C - O(log(n)),S.C - O(1)

// Optimised approach

const countDigitsOptimal = (n) => {
  if (n === 0) return 1;
  let logBase10 = Math.log10(Math.abs(n));
  return Math.floor(logBase10 + 1);
};
const digitsOptimal = countDigitsOptimal(1221);
console.log(digitsOptimal);

// TC - O(1) , SC - O(1)

// Optimised Approach -Javascript based(Converting to string)

const countDigitsString = (n) => {
  let strDigit = n.toString();
  return strDigit.length;
};

const digitsJS = countDigitsString(1221);
console.log(digitsJS);

// T.C - O(1) , S.C - O(1)

// 2) Given an integer N return the reverse of the given number.Note - The number can be negetive.
// Solving it using extraction algo
const reverseNumber = (n) => {
  let revNum = 0;
  let originalNum = n;
  n = Math.abs(n);
  while (n > 0) {
    let lastDigit = n % 10;
    n = Math.floor(n / 10);
```

```
    revNum = revNum * 10 + lastDigit;
  }
  //   Restore the negetive sign for negetive numbers
  if (originalNum < 0) {
    revNum = -revNum;
  }
  return revNum;
};
const reverseNum = reverseNumber(-4321);
console.log(reverseNum);

// T.C - O(log10(n)) , S.C - O(1)

// 3) Check if a number is Palindrome or Not(Usage of Extraction Algo)

const checkPalindromeNumber = (n) => {
  let originalNum = n;
  let revNum = 0;
  if (n < 0 || (n !== 0 && n % 10 === 0)) {
    return false;
  }
  while (n > 0) {
    let lastDigit = n % 10;
    n = Math.floor(n / 10);
    revNum = revNum * 10 + lastDigit;
  }
  if (originalNum === revNum) {
    return "The number is Palindrome";
  } else {
    return "The number is not Palindrome";
  }
};
const palindromeNumber = checkPalindromeNumber(1221);
console.log(palindromeNumber);

// TC - O(log10(n)) , SC - O(1)

// 4) Given an integer N, return true it is an Armstrong number otherwise return false.
// An Amrstrong number is a number that is equal to the sum of its own digits each raised to
the power of the number of digits.
// We will again use our extraction algorithm to solve this problem

const armStrongNumber = (n) => {
  if (n < 0) {
    return "Negetive numbers are not ArmStrong";
  }
  let originalNum = n,
    temp = n;
  // counting the number of digits
  let countDigit = 0;
  while (temp > 0) {
    countDigit++;
    temp = Math.floor(temp / 10);
  }
```

```
  temp = n;
  // Calculate the armstrong number
  let sum = 0;
  while (temp > 0) {
    let lastDigit = temp % 10;
    sum += Math.pow(lastDigit, countDigit);
    temp = Math.floor(temp / 10);
  }
  if (sum === originalNum) {
    return "The number is Armstrong";
  } else {
    return "The number is not Armstrong";
  }
};
const checkArmStrong = armStrongNumber(153);
console.log(checkArmStrong);

// TC - O(log10(n)) , SC - O(1)

// 5) Given an integer N, return all divisors of N.
// Brute force Approach :-

// Divisors are those numbers which when divided by a particular number results in the remainder 0.
// So we can say divisors will always be between (1 --- N)
// Hence we can obtain their values by (N%i) == 0 and then pushing them into an array
const printDivisors = (n) => {
  const divisors = [];
  for (let i = 0; i <= n; i++) {
    if (n % i === 0) {
      divisors.push(i);
    }
  }
  return divisors;
};
const divArray = printDivisors(36);
console.log(divArray);
// TC - O(n) SC- O(divisors) i.e. O(d) where d is the length of divisors

// Optimised Approach
// Optimised approach can be solved by using divisor algorithm
const printDivisorsOptimised = (n) => {
  let divisors = [];
  for (let i = 1; i * i <= n; i++) {
    if (n % i === 0) {
      divisors.push(i);
      if (n / i !== i) {
        divisors.push(n / i);
      }
    }
  }
  divisors.sort((a, b) => a - b);
  return divisors;
};
```

```
const divArrayOptimised = printDivisorsOptimised(36);
console.log(divArrayOptimised);

// TC - O(square root(N)), SC - O(d) where d is the number of divisors


```